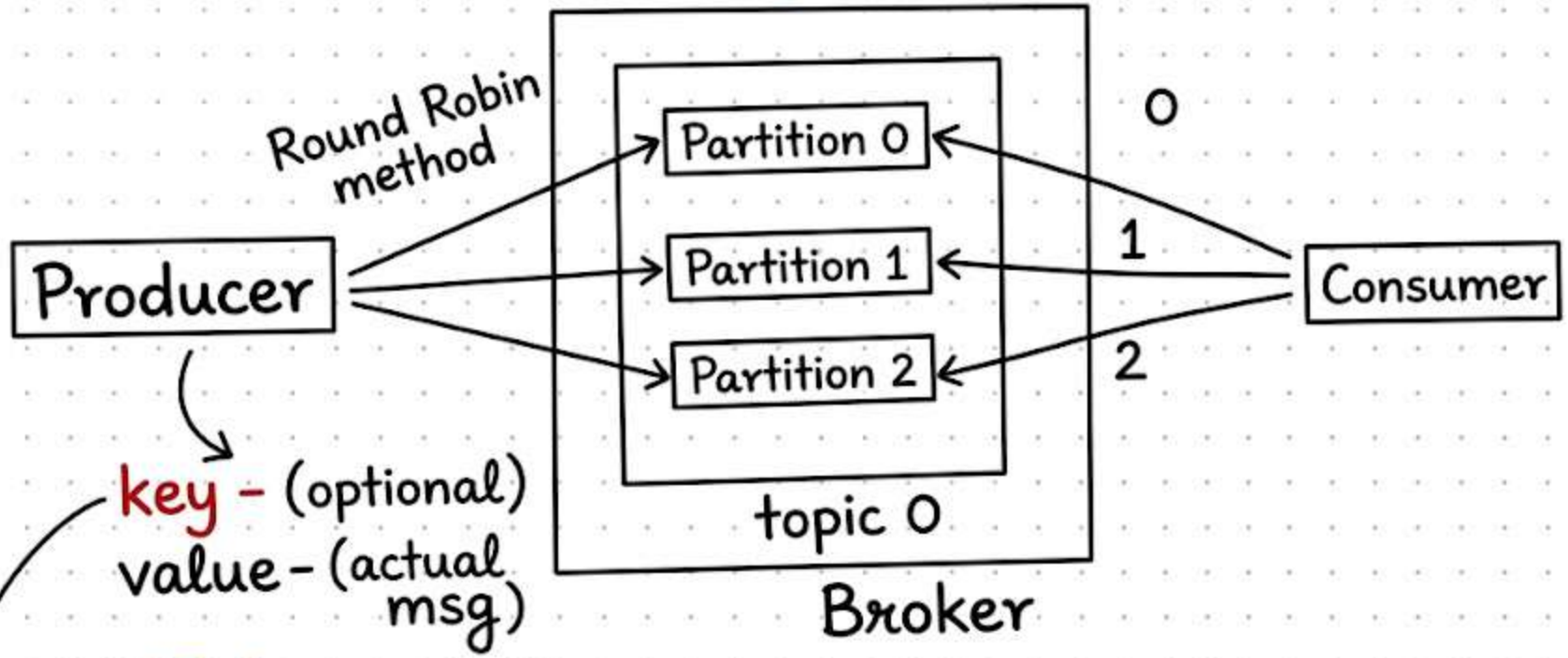
# Lecture 09: Kafka Part-2 | Caching Techniques

## Cache Eviction policies



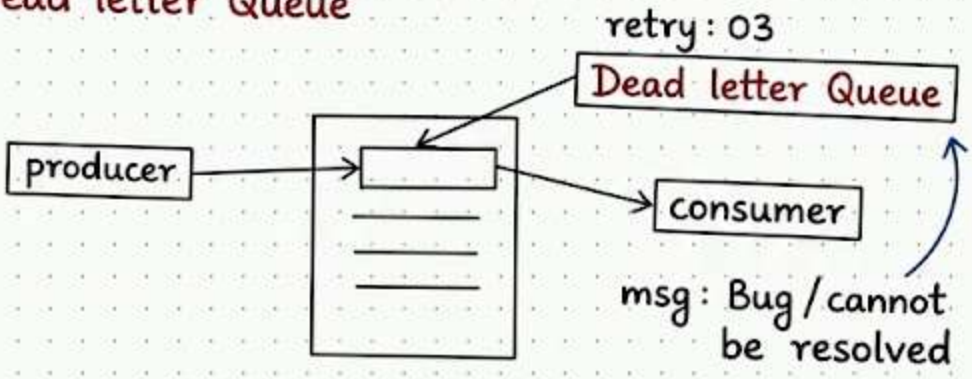

key - (optional)
value - (actual msg)

Topic - topic 0

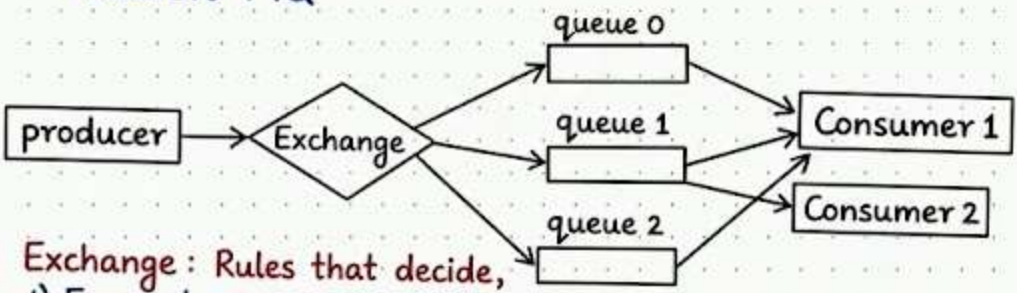
partition - (optional)

hash % (no. of partition) - if key is given

## Dead letter Queue




## Rabbit MQ




Exchange : Rules that decide, msg will go in which queue.

1) Fan-out → Broadcast
2) Direct → match Message key to RK.
3) Topic Exchange → wildcards.

keys:
↳ Message key : Key that comes with the mssg
↳ Routing key : exchange key

→ Topic exchange : Wildcards are allowed. (*)

( ( MK : 123qty
RK : 123* )

↳ Dont directly match, but map with inital character before * . like all starts with 123

→ Re-Queue mechanism (dead letter)
↳ kafka : pull based technique (consumer polling)
↳ RabbitMQ : Push based technique

# Caching

## Caching

Cache : faster accessible unit, temporary storage (TTL),

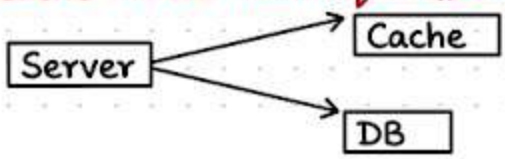
### Types of Cache :

CDN → caching use (static resource)
Load Balancer (web pages)
Server side caching (Redis)
proxies also use caching.

→ Distributed Caching → ◉ Data consistency
* C1 C2 C3
◉ internally uses consistent hashing (for scaling)

→ Cache Data Post / Get

Algo 1 :
1. Cache Aside technique (~~Red~~ Retrieve)

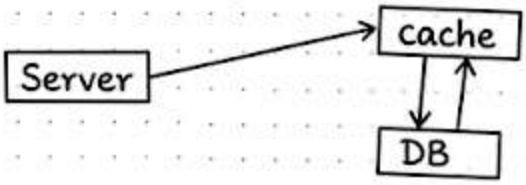



1. check cache
2. if found (cache hit) return the data
3. if not, (cache miss) fetch the data from DB, push it into the cache and then return the data

Pros :
↳ Simple to implement
↳ DB doc structure and cache can be different

Cons :
↳ For every new data there will be always be cache miss
↓
pre-heating the cache

## 2. Read through cache: (Data Retrieval)

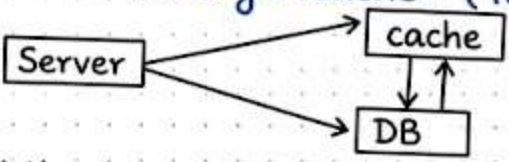



1. Read through cache
2. if found, cache hit, return the data
3. Cache miss, cache internal library will automatically fetch the data from DB and put if in cache and return to the server

pros:
Client don't need to bother about DB logic

cons:
↳ Cache doc structure should be same as DB
↳ starting → cache miss
↓
pre-heat the cache

## 3. Write through cache: (To post data)

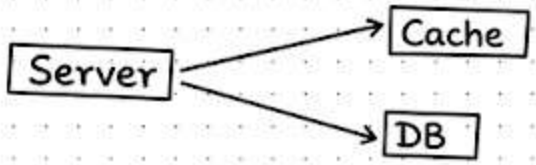



Write to cache and DB simultaneously

pros:
Consistency will be high

cons:
↳ Slow
↳ Uses 2 phase commit (overhead)

## 4. Write back / behind cache :




write into cache and async write to DB

Disadvantage : Inconsistency can come

Pros :
fast

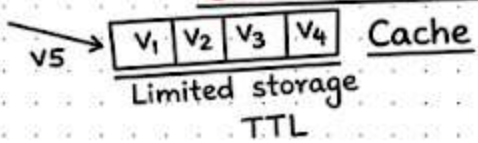
## 5. Write Around technique :




Directly write Data to DB
Do not write data to Cache, just invalidate the data in cache (mark it as dirty)

Advantage :
without using any 2PL (phase locking), it can maintain consistency.

## Cache Eviction policies

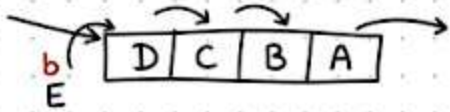



How to and what to remove data from cache

LRU Cache (Least Recently used)
MRU Cache (Most recently used)
LFU LFU (Least frequently used)
FIFO (First In First Out)
Random

## LRU (Least Recently Used)

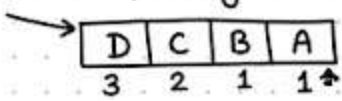
When the cache is full, LRU evicts the item that hasn't been accessed in the longest time.



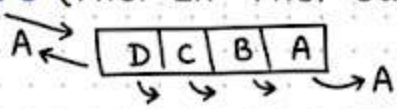

pros: Simplicity
cons: Assume past record patterns will repeat in future

MRU (Most Recently Used) – replace something that most recently used.
Disadvantage: Do not bother about past records

LFU (Least Frequently used)




FIFO (First In - First Out)




pros: Simplicity
Do not need to maintain any timestamp or frequency

Cache: Cache miss will be higher

Random Cache Eviction
Remove and add randomly.

Client side Caching → Cookies